

Homework 3

[2026-1] 데이터사이언스를 위한 컴퓨팅 (001)

Due: 2026년 5월 26일 23:59

1. Max-heap with Tree-based Structure [25pts]

Implement a binary max-heap with tree-based structure which has the following properties:

- Structure property: It is a complete binary tree.
- Heap property: The value of every non-root node is equal to or smaller than the value of its parent.

The binary max-heap should use linked `Node` instances with pointers rather than an array-based indexing approach.

You need to implement the `enqueue` and `dequeue` method in `MaxHeap` class provided in `functions.cpp`.

- The `enqueue` method takes an integer `val` as input and returns `void`. It creates a new node with the value and appends this node to the heap as the last element. After adding the new node, the method restores the heap property.
- The `dequeue` method takes no input and returns `void`. It removes the value at the root node, which is the **maximum** value in the heap. The root node is then replaced with the value of the last node, and the last node is removed from the heap. Finally, the method restores the heap property.
- Ensure that each operation is completed in $O(\log n)$ time.

The methods below are provided, each described as follows:

- `int top()`: Returns the maximum value of the heap.
- `int size()`: Returns the size of the heap.
- `void print()`: Prints the values of all nodes in the heap from the root to the leaves, traversing each level from left to right. It is also called breadth-first search (BFS).

A helpful method to guide you is:

- `Node* getNodeByIndex(Node* root, int index)`: Given the root node of a heap (represented as a complete binary tree) and a logical 1-based index, this function traverses the tree to locate the corresponding node. It returns a pointer (`Node*`) to the node at that specific index.
 - e.g. `root` has index 1, `root->left` has index 2, ..

Implementation of `getNodeByIndex` is completely optional. You may or may not choose to implement it as part of your solution.

Example

```
MaxHeap heap;

heap.enqueue(30);
heap.enqueue(10);
heap.enqueue(100);
heap.enqueue(40);
heap.enqueue(50);
std::cout << heap.top() << std::endl; // 100
heap.print();                          // 100 50 30 10 40

heap.dequeue();
std::cout << heap.top() << std::endl; // 50
heap.print();                          // 50 40 30 10
```

2. Minimum Distance [25pts]

Given an array of points representing integer coordinates on a 2D plane, where each point is represented as (x_i, y_i) , your goal is to find the minimum distance required to connect all points. The distance between any two points (x_i, y_i) and (x_j, y_j) is calculated using the Manhattan distance: $|x_i - x_j| + |y_i - y_j|$.

Implement the `minDistance` function in `functions.cpp` following the instructions below.

2.1. Input

- `std::vector<std::vector<int>>& points`: A reference to a vector of integer pairs, where each pair represents the (x, y) coordinates of a point on a 2D plane.

2.2. Output

- The `minDistance` returns an integer representing the minimum distance required to connect all points.

2.3. Constraints

- $1 \leq \text{points.size()} \leq 100$
- $-10^6 \leq x_i, y_i \leq 10^6$

Example

```
std::vector<std::vector<int>> points;

points.push_back(std::vector<int>({0, 0}));
points.push_back(std::vector<int>({1, 0}));
points.push_back(std::vector<int>({1, 2}));

std::cout << minDistance(points) << std::endl; // 3
```

3. Validate Red-Black Tree [50pts]

A Red-Black Tree is a type of binary search tree (BST) that maintains balance and performance guarantees through specific properties. A Red-Black Tree consists of nodes, which have the following attributes:

```
int val;  
bool isRed;  
Node* left;  
Node* right;
```

Implement the `validateRedBlackTree` function in `functions.cpp` to verify if a given tree is a valid Red-Black Tree. In a Red-Black Tree, all leaves (represented as `nullptr`) are considered BLACK nodes by definition.

3.1. Input

- `Node* root`: A pointer to the root node of the tree.

3.2. Output

- Returns `true` if the tree is a valid Red-Black Tree, otherwise returns `false`.

Example

```
Node* root = new Node(5, false);  
std::cout << validateRedBlackTree(root) << std::endl; // 1 (true)  
delete root;
```

Submission Instructions

- For each problem, write your functions in the designated `functions.cpp` file according to the instructions. You may modify other files for testing purposes, but keep in mind that **only** `functions.cpp` **will be graded**, using the original header file `functions.hpp` and `main.cpp`.
- Grading will be conducted using C++20 standard. The `main` function in the autograder, which includes test cases, may differ from the `main` function released to students. The compilation process is as follows:

```
For all problems,  
$ g++ -c main.cpp -o main.o -std=c++20 (using the original main.cpp)  
$ g++ -c functions.cpp -o functions.o -std=c++20 (using your functions.cpp)  
$ g++ main.o functions.o -o main  
./main
```

- When submitting your codes, ensure that all test codes printing out `stdout` are removed, unless explicitly instructed otherwise. **Grading relies solely on the `stdout`, and including any unrelated codes may result in inaccurate grading.**
- You must **NOT** share your code with other students. Additionally, students must not rely on LLMs to generate code directly. Any student found to have a high level of code similarity, as determined by our similarity detection process, may face serious penalties.
- If you submit late, grace days will be automatically deducted. You have 5 grace days available for homework submissions throughout the semester. Grace days are counted in 24-hour increments from the original due time. For example, if an assignment is due on 1/1 at 11:59 PM, submitting it 30 minutes late will use one grace day, while submitting it on 1/3 at 9:00 PM will use two grace days. Late submissions will **NOT** be accepted once all grace days have been used.
- For questions about this homework, use the “Homework 3” discussion forum on eTL.
- Failing to adhere to the submission guidelines may result in penalties applied without exception.